

Q-COMMUTE

Quantum-Inspired Multi-Objective Urban Traffic Optimization Engine

DOCUMENT TYPE	DOMAIN	GRAPH TARGET	CORE ALGORITHM
SIH Complete Master Dossier	Smart City & Intelligent Mobility	OpenStreetMap Central Delhi	Quantum PSO (QPSO) + Delta Well

1. Executive Overview & Problem Statement

Modern metropolitan transportation networks suffer from systemic gridlocks, localized vehicle density spikes, and sudden disruptions (road closures, construction, VIP movement, accidents). Standard commercial navigation algorithms (such as classical Dijkstra, A*, or greedy heuristics) solve single-objective static shortest paths (typically distance or free-flow time).

Key Limitations of Current Navigation Systems:

- Single-Objective Myopia:** Standard routers prioritize distance over dynamic congestion volatility, causing thousands of commuters to be routed into the exact same bottleneck corridor.
- NP-Hard Dynamic Multi-Objective Complexity:** Solving multi-criteria routing (balancing travel time, distance, and congestion penalties simultaneously) across large urban graphs (10,000+ edges) in real-time creates a severe computational bottleneck.
- Local Optima Trapping:** Classical greedy heuristics get stuck in local sub-optimal routes when dynamic road closures occur.

The Q-Commute Solution: A quantum-inspired urban traffic management system that maps Delhi's real road graph (6,626 nodes, 16,284 edges) and computes Pareto-optimal multi-objective commuter paths using Quantum-Behaved Particle Swarm Optimization (QPSO) with delta-potential well wave mechanics.

2. Step-by-Step: How We Built Q-Commute From Scratch

The development of Q-Commute followed a rigorous 7-step engineering pipeline:

Step 1: Geospatial Road Graph Acquisition & Cleaning

- Extracted roadway networks for Central Delhi using `osmnx` and converted them into a directed multigraph (`networkx.MultiDiGraph`).
- Cleaned topological anomalies, isolated components, and dead-ends to establish strongly connected routing topologies.
- Assigned physical metrics to every edge: segment length in metres, base speed limits in km/h, and free-flow traversal times in seconds.

- Indexed all 6,626 intersection vertices with real GPS latitude/longitude coordinates (`WGS-84 / EPSG:4326`).

Step 2: Multi-Objective Scalar Cost Formulation

- Formulated the normalized multi-criteria objective function:

$$\min F(p) = W_t \cdot [T(p) / T_{ref}] + W_d \cdot [D(p) / D_{ref}] + W_c \cdot [C(p)]$$

- Integrated topological constraints:
 - Endpoint Continuity:** Route path must begin at origin and terminate at destination (`v_0 = src, v_n = dst`).
 - Edge Adjacency:** Consecutive vertices must represent valid directed edges (`(v_i, v_{i+1}) ∈ E(G)`).
 - Incident Avoidance:** Strict exclusion of blocked edges (`e_k ∉ Closed(E)`).

Step 3: Algorithmic Engine Implementation

- QPSO Core:** Implemented quantum particle swarm optimization with delta-potential well wave functions, Mean Best (`m_best`) coordinate calculation, and Monte Carlo sampling.
- Continuous-to-Discrete Graph Decoder:** Built a priority-guided node walker that maps continuous quantum particle position vectors into valid, continuous topological road sequences.
- Dijkstra Baseline:** Implemented deterministic shortest path algorithm for exact baseline verification.
- Genetic Algorithm (GA):** Built evolutionary benchmark with single-point crossover, random mutations, and path repair mechanisms.

Step 4: Live Dynamic Traffic Simulation Engine

- Engineered an in-memory mutable graph state manager that dynamically applies congestion multiplier penalties (1.0x to 5.0x), roadwork delays, and complete road closures.
- Implemented reactive real-time rerouting triggered immediately upon user incident injection on the map.

Step 5: SciPy Analytical Modeling & Matplotlib Reporting

- Integrated `scipy.optimize.curve_fit` to model the swarm's non-linear exponential decay convergence: `y = a · exp(-b · t) + c`.
- Extracted theoretical asymptotic bounds (`c`) to mathematically verify numerical convergence stability.
- Developed server-side headless Matplotlib report generation exported as downloadable publication-ready plots.

Step 6: SQLite Persistence Layer

- Designed an embedded relational database (`routes.db`) using SQLite to record historical routing runs, coordinate geometries, algorithm tags, costs, and server calculation latencies.

Step 7: Production Frontend & Map Dashboard

- Built a responsive Single Page Application (SPA) using React, Vite, TailwindCSS, DaisyUI, Leaflet, and Chart.js.

- Styled formulas, badges, convergence charts, 3-way benchmark comparison tables, and map layer controls.

3. Technology Stack & Architectural Rationale

LAYER	TECHNOLOGY SELECTED	WHY THIS TECHNOLOGY? (OVER ALTERNATIVES)
Backend Framework	FastAPI (Python 3.10+)	Asynchronous ASGI runtime (asyncio), automatic OpenAPI/Swagger documentation, strict Pydantic type safety, and minimal latency compared to heavier Django/Flask frameworks.
Graph Topology	NetworkX & OSMnx	Industry-standard mathematical graph modeling, seamless OpenStreetMap geospatial ingestion, and efficient topological traversal routines.
Scientific Math	NumPy & SciPy	Vectorized C-speed matrix calculations, fast random uniform sampling for quantum wave equations, and non-linear least squares regression (<code>curve_fit</code>).
Visual Plotting	Matplotlib	Headless server-side publication-grade analytical plot generation, returning base64 images directly to the client.
Persistence	SQLite (Native stdlib)	Zero-configuration, embedded ACID relational database with zero network latency, eliminating external database server dependencies.
Frontend Engine	React.js + Vite	Declarative virtual DOM for smooth map animations, instant sub-second hot reloading, and modular component hierarchy.
Mapping Engine	Leaflet & React-Leaflet	Ultra-lightweight (42KB), hardware-accelerated tile rendering, zero API key constraints, and zero Google Maps billing overhead.
UI & Styling	TailwindCSS & DaisyUI	Utility-first CSS compiling to lean production bundles, consistent design system, accessible color contrast, and custom responsive layouts.
Live Charting	Chart.js	HTML5 Canvas-based real-time line and bar charts for convergence trajectories and multi-algorithm benchmark comparisons.

4. Complete Mathematical Formulations & Quantum Mechanics

The Multi-Criteria Objective Function

The total objective cost of a commuter route `p = (v_0, v_1, ..., v_n)` is computed as:

$$\min F(p) = W_t \cdot [T(p) / T_{ref}] + W_d \cdot [D(p) / D_{ref}] + W_c \cdot [C(p)]$$

Where:

- $T(p) = \sum_{e \in p} [\text{length}(e) / (\text{speed}(e) \cdot \mu_{\text{congestion}}(e))]$ (Total Travel Time)
- $D(p) = \sum_{e \in p} \text{length}(e)$ (Total Distance in Metres)
- $C(p) = \sum_{e \in p} (\mu_{\text{congestion}}(e) - 1.0)$ (Cumulative Congestion Penalty Index)
- T_{ref}, D_{ref} : 75th-percentile normalization scaling constants across graph paths.
- $W_t + W_d + W_c = 1.0$ (User Preference Weights)

Quantum-Behaved Particle Swarm Optimization (QPSO) Update Rules

In standard classical PSO, particles have Newtonian velocities that can cause particles to overshoot global optima or get trapped in local basins. In **QPSO**, particles move in a quantum delta-potential well centered at the local attractor p_i .

The probability density function is derived from the Schrödinger wave equation:

$$Q(x) = |\psi(x)|^2 = (1 / L) \cdot \exp(-2 \cdot |p_i - x| / L)$$

Using Monte Carlo inverse-transform sampling, the position update becomes:

$$x_i(t+1) = p_i \pm \beta \cdot |m_{\text{best}} - x_i(t)| \cdot \ln(1 / u), \text{ where } u \sim \text{Uniform}(0, 1)$$

1. Mean Best (m_{best}): The center of gravity of personal best positions:

$$m_{\text{best}} = (1 / M) \cdot \sum_{i=1}^M P_{i,\text{best}}$$

2. Local Attractor (p_i):

$$p_i = \varphi \cdot P_{i,\text{best}} + (1 - \varphi) \cdot G_{\text{best}}, \text{ where } \varphi \sim \text{Uniform}(0, 1)$$

3. Contraction-Expansion Coefficient (β): Linearly decays across iterations:

$$\beta(t) = \beta_{\text{max}} - (t / G) \cdot (\beta_{\text{max}} - \beta_{\text{min}}) = 1.0 - 0.6 \cdot (t / G)$$

SciPy Analytical Curve Fitting

Swarm convergence is verified using non-linear least squares regression:

$$F_{\text{fit}}(t) = a \cdot \exp(-b \cdot t) + c$$

- a : Initial improvement potential amplitude.
- b : Convergence decay rate coefficient.
- c : SciPy Asymptotic Bound (theoretical optimum limit as $t \rightarrow \infty$).

5. Complete UI Guide & Operation Workflow

1. **Origin & Destination Selection:** In the left *Route Planner* panel, pick an Origin (e.g., *Connaught Place*) and Destination (e.g., *India Gate*).
2. **Hyperparameter & Weight Tuning:**

- Set **Swarm Particles** (default: 20) and **Iterations** (default: 40).
 - Adjust **Multi-Objective Weights**: Time (`w_t`), Distance (`w_d`), and Congestion (`w_c`).
3. **Execute Route Optimization**: Click **Optimize Route (QPSO)**.
- Map renders the optimized purple polyline.
 - **Metrics Tab**: Displays travel distance, time, objective score, and computation latency.
 - **Quantum Tab**: Displays real-time cost convergence curve and SciPy curve parameters.
4. **Simulate Live Incidents**:
- In the *Dynamic Traffic Simulation* card, select an incident type (*Congestion*, *Roadwork*, or *Closure*).
 - Click on any roadway on the map.
 - The system mutates the road weight in real-time, displays the blocked segment, and recalculates an adaptive detour.
5. **3-Algorithm Comparative Benchmark**:
- Click **Run All 3 Algorithms Benchmark**.
 - Open the **Benchmark Tab** to inspect the dual-axis chart (Objective Cost vs Travel Time) and performance table (QPSO vs. Dijkstra vs. GA).
6. **Inspect Mathematical Formulas & History**:
- Switch to the **Formula Tab** for mathematical proofs and NetworkX graph metrics.
 - Switch to the **History Tab** to inspect persistent SQLite run records.
 - Click **Matplotlib Report** in the header for downloadable analytical plots.

6. Exhaustive UI Glossary (Every Word on the Website)

CATEGORY	TERM VISIBLE ON UI	EXACT MEANING & TECHNICAL DEFINITION
Header	Q-COMMUTE	Project title: Quantum-Inspired Commuter Routing Engine.
Header	Quantum-Inspired Traffic Optimization	Core methodology: applying quantum wave equations to classical combinatorial routing.
Header	Matplotlib Report	Button generating server-side analytical convergence and benchmark plots.
Route Planner	OSM Delhi	Indicates the underlying OpenStreetMap dataset for Central Delhi.
Route Planner	Swarm Particles (M)	The number of candidate route solutions exploring the graph in parallel (default: 20).
Route Planner	Iterations / Generations (G)	The number of discrete quantum position update cycles executed by the swarm (default: 40).
Route Planner	Multi-Objective Weights	User priority sliders balancing Time (<code>W_t</code>), Distance (<code>W_d</code>), and Congestion (<code>W_c</code>).
Simulation	Incident Type	Disturbance class: Congestion (speed multiplier), Roadwork (delay), Closure (complete cut).
Simulation	Live Injection	Real-time in-memory graph edge mutation without server restart.
Route Legend	QPSO (Quantum) [Adaptive]	Quantum PSO route solution rendered in purple.
Route Legend	Dijkstra (Exact) [Baseline]	Deterministic shortest path solution rendered in green.
Route Legend	GA (Evolutionary) [Heuristic]	Genetic algorithm route solution rendered in orange.
Route Legend	Closed Road [Blocked]	Edges disabled due to simulated incidents rendered in red.
Metrics Tab	Objective Cost $F(p)$	Dimensionless scalar score combining normalized time, distance, and congestion (lower is better).
Metrics Tab	Runtime (Latency)	Server computation time in milliseconds (ms) for the routing query.
Metrics Tab	Route Nodes	Count of physical intersection vertices comprising the generated path.
Quantum Tab	Convergence Trajectory	Line chart plotting global best objective cost against iteration number <code>t</code> .
Quantum Tab	SciPy Curve Fitting & Statistics	Non-linear regression analysis using <code>scipy.optimize.curve_fit</code> .

CATEGORY	TERM VISIBLE ON UI	EXACT MEANING & TECHNICAL DEFINITION
Quantum Tab	Relative Optimization	Percentage improvement between initial random swarm best and final converged cost.
Quantum Tab	SciPy Asymptotic Bound	Theoretical mathematical limit <code>c</code> towards which the cost function converges as <code>t → ∞</code> .
Formula Tab	Nodes ($ V $) / Edges ($ E $)	Total graph size: 6,626 intersection nodes and 16,284 directed road segments.
Formula Tab	Graph Density	Ratio of existing edges to maximum possible edges (0.000371).
Formula Tab	Avg Node Degree	Average number of road connections per intersection (2.46).
Formula Tab	Spatial Coordinate System	WGS-84 (EPSG:4326) standard GPS latitude and longitude system.
History Tab	SQLite Persistence Engine	Embedded database logging all historical routing calculations.
History Tab	Total Runs Saved	Total count of routing executions recorded in <code>routes.db</code> .

7. SIH Presentation Defense: Exhaustive Cross-Questions & Answers

ALGORITHM & QUANTUM ARCHITECTURE

Q1: Is this running on an actual physical quantum computer (e.g. IBM Qiskit / D-Wave)?

Answer: "No, and that is an intentional and practical engineering decision. Q-Commute uses a **Quantum-Inspired Metaheuristic (QPSO)**. Physical Quantum Processing Units (QPUs) in the current NISQ era suffer from severe quantum decoherence, require cryogenic cooling, have cloud queue wait times of several minutes, and cannot support sub-second commuter GPS queries. QPSO models the mathematical mechanics of quantum wave functions and delta-potential wells on classical cloud/edge hardware, achieving global multi-objective optimization with sub-second execution speeds and zero specialized hardware dependencies."

COMPUTATIONAL THEORY

Q2: Why not simply use Dijkstra or A* for multi-objective routing?

Answer: "Dijkstra and A* are exact deterministic single-criterion algorithms. In urban traffic, routing is inherently multi-objective (balancing travel time, distance, and congestion volatility simultaneously). Multi-objective shortest path problems are NP-hard; running exact multi-criteria Dijkstra causes combinatorial explosion on large urban networks. QPSO efficiently explores the continuous Pareto frontier in bounded polynomial time $O(G \cdot M)$, avoiding local minima while finding high-quality balanced routes in under 2 seconds."

REPRESENTATION & MAPPING

Q3: How do you map continuous quantum particle coordinates into discrete road paths?

Answer: "We use a **priority-guided continuous-to-discrete graph decoder**. Each coordinate of a particle vector corresponds to a continuous traversal priority weight. When building a candidate path, a graph-walking heuristic traverses from origin to destination, selecting the highest-priority adjacent valid edge in the NetworkX graph. This strictly enforces road network continuity constraints without generating disconnected subgraphs."

MATHEMATICAL VERIFICATION

Q4: How do you mathematically prove that QPSO actually converged and did not terminate randomly?

Answer: "We apply **SciPy Non-Linear Least Squares Curve Fitting** (`scipy.optimize.curve_fit`) to the convergence trajectory data. By fitting the iteration curve to an exponential decay model $F(t) = a \cdot \exp(-b \cdot t) + c$, we compute the asymptotic bound `c`. When the final cost is within numerical tolerance of `c`, we have mathematical confirmation of stable convergence."

REAL-WORLD SCALABILITY

Q5: Can Q-Commute scale to an entire state or country?

Answer: "Yes, through **hierarchical graph partitioning and contraction hierarchies**. For a metropolitan area like Delhi (16,000+ edges), Q-Commute runs in under 2 seconds on a single CPU core. For state or national scale, the network is partitioned into regional boundary transit graphs. Inter-city routes are solved on the macro-graph, while intra-city segments are optimized via localized QPSO swarms."

GOVERNMENT & CIVIC IMPACT

Q6: What is the practical value of Q-Commute for Smart City Authorities and Traffic Police?

Answer: "Three key benefits: 1. **Network Load Balancing:** Prevents 'flash mob' congestion caused by commercial GPS routing all commuters down the same single street. 2. **Emergency Vehicle Corridors:** Enables green-corridor routing for ambulances during live road blockages. 3. **Emission Reduction:** Multi-objective balance between distance and stop-and-go congestion directly reduces commuter idle emissions."